PS-12

Blockchain-Based Vehicle Ownership and Service History

Project Report

Developed using Solidity, Remix IDE, MetaMask and a Web Frontend

Blockchain Project • PS-12

Sahas Bochare 08

1. Problem Statement

When purchasing a used vehicle, buyers may not have access to reliable information about previous ownership, servicing and maintenance. Service documents may be incomplete, difficult to verify, or controlled by different parties.

PS-12 proposes a blockchain-based vehicle history system that maintains a transparent and tamper-resistant record of vehicle ownership and service activities. The system allows an authorised administrator to register vehicles, authorised service centres to add service records, owners to transfer ownership, and users to search for and view a vehicle's history.

2. Proposed Solution

The proposed solution is a decentralized vehicle-history application backed by a Solidity smart contract. Vehicle information, ownership changes and service records are stored on a blockchain network rather than in a conventional centralized database.

- Each vehicle is identified by a unique vehicle ID.
- The system stores the registration number, model, manufacturer and current owner.
- Ownership transfers are recorded with the previous and new owner addresses.
- Only authorised service-centre addresses can add service records.
- Service records contain a timestamp, description and service-centre wallet address.
- Users can retrieve vehicle details, ownership history and service history.
- The frontend provides a user-friendly interface for interacting with the smart contract.
- A public vehicle-history page can be generated for a particular vehicle.

3. Project Objectives

- Build a blockchain-based record of vehicle ownership.
- Maintain a permanent history of service activities.
- Prevent ordinary users from editing or deleting previous service records.
- Restrict service-record creation to authorised service centres.
- Make ownership transfer possible through a smart contract.
- Allow users to verify a vehicle's registered information and history.
- Provide a frontend through which users can interact with the deployed contract.
- Demonstrate the system on a blockchain test network.

4. Blockchain Concepts Used

Concept	Use in Project	Benefit
Smart Contract	Solidity contract controls registration, ownership and service records.	Rules are enforced by code.
Decentralization	Records are stored on a blockchain network instead of one central database.	Reduces dependence on a single authority.
Immutability	Past service and ownership records are retained on-chain.	Improves trust and auditability.
Wallet Address	Vehicle owners and service centres	Provides a verifiable identity for

	are represented by blockchain addresses.	transactions.
Transaction	State-changing operations are submitted as blockchain transactions.	Creates an auditable history of changes.
Timestamp	Service and ownership records store blockchain timestamps.	Shows when an event was recorded.
Access Control	Admin and authorised service-centre checks restrict sensitive functions.	Prevents unauthorised updates.

5. Smart Contract Explanation

The core logic is implemented in a Solidity smart contract named VehicleHistory. The contract maintains vehicle details, service records, ownership history and authorised service-centre addresses.

5.1 Main Smart Contract Functions

Function	Purpose	Access / Type
registerVehicle()	Registers a vehicle with its ID, registration number, model, manufacturer and owner.	State-changing
authorizeServiceCenter()	Adds a wallet address to the list of authorised service centres.	Admin-controlled
addServiceRecord()	Adds a service date, description and service-centre address to a vehicle's history.	Authorised service centre
transferOwnership()	Transfers a vehicle to a new owner and records the ownership change.	Current owner
getVehicleDetails()	Returns the stored vehicle details and current owner.	Read-only
getServiceHistory()	Returns all stored service records for a vehicle.	Read-only
getOwnershipHistory()	Returns the ownership history for a vehicle.	Read-only
vehicleExistsCheck()	Checks whether a vehicle ID exists in the contract.	Read-only
authorizedServiceCenters()	Checks whether a particular address is authorised as a service centre.	Read-only
admin()	Returns the administrator address.	Read-only

5.2 Data Structures

- Vehicle details: vehicle ID, registration number, model, manufacturer and owner address.
- ServiceRecord: service date, service description and service-centre address.
- OwnershipRecord: owner address and transfer timestamp.
- Authorised service-centre mapping: wallet address → authorised/not authorised.
- Vehicle existence mapping: vehicle ID → whether the vehicle is registered.

5.3 Access Control

The contract uses access checks to protect state-changing operations. During testing, attempting to add a service record from a non-authorised address resulted in the expected revert message, 'Not an authorized service center'. After the service-centre address was authorised, the same operation succeeded. Similarly, ownership transfer is restricted to the current owner.

6. Application Architecture

The application follows a simple blockchain dApp architecture. The browser-based frontend communicates with the deployed Solidity contract through the user's wallet. MetaMask is used to connect the user's account and approve blockchain transactions.

Layer	Technology / Component	Responsibility
User Interface	HTML / CSS / JavaScript	Forms, buttons, vehicle history display and user interaction.
Web3 Integration	JavaScript + Contract ABI	Calls smart-contract functions and reads blockchain data.
Wallet	MetaMask browser extension	Provides account address and signs transactions.
Smart Contract	Solidity / VehicleHistory	Enforces business rules and stores vehicle history.
Blockchain	Ethereum-compatible test network	Persists contract state and transaction history.
Development / Testing	Remix IDE	Compilation, deployment and contract-function testing.

User → Frontend → MetaMask → Blockchain Transaction → VehicleHistory Smart Contract

↓

Vehicle / Ownership / Service History

7. Application Workflow

Vehicle Registration: The administrator registers a vehicle with a unique vehicle ID and basic vehicle information.

Service Centre Authorization: The administrator authorises a service-centre wallet address.

Service Record Addition: The authorised service centre enters a vehicle ID and service description. The transaction stores the record on-chain.

Ownership Transfer: The current owner submits a transfer transaction with the new owner's wallet address.

History Verification: A user searches for the vehicle and retrieves its details, ownership history and service history.

Public History: The application can generate a vehicle-specific public history page using the vehicle ID.

8. Testing and Demonstration

The smart contract was compiled and tested in Remix IDE. Testing covered the main functional requirements and also verified that access-control rules correctly reject unauthorised operations.

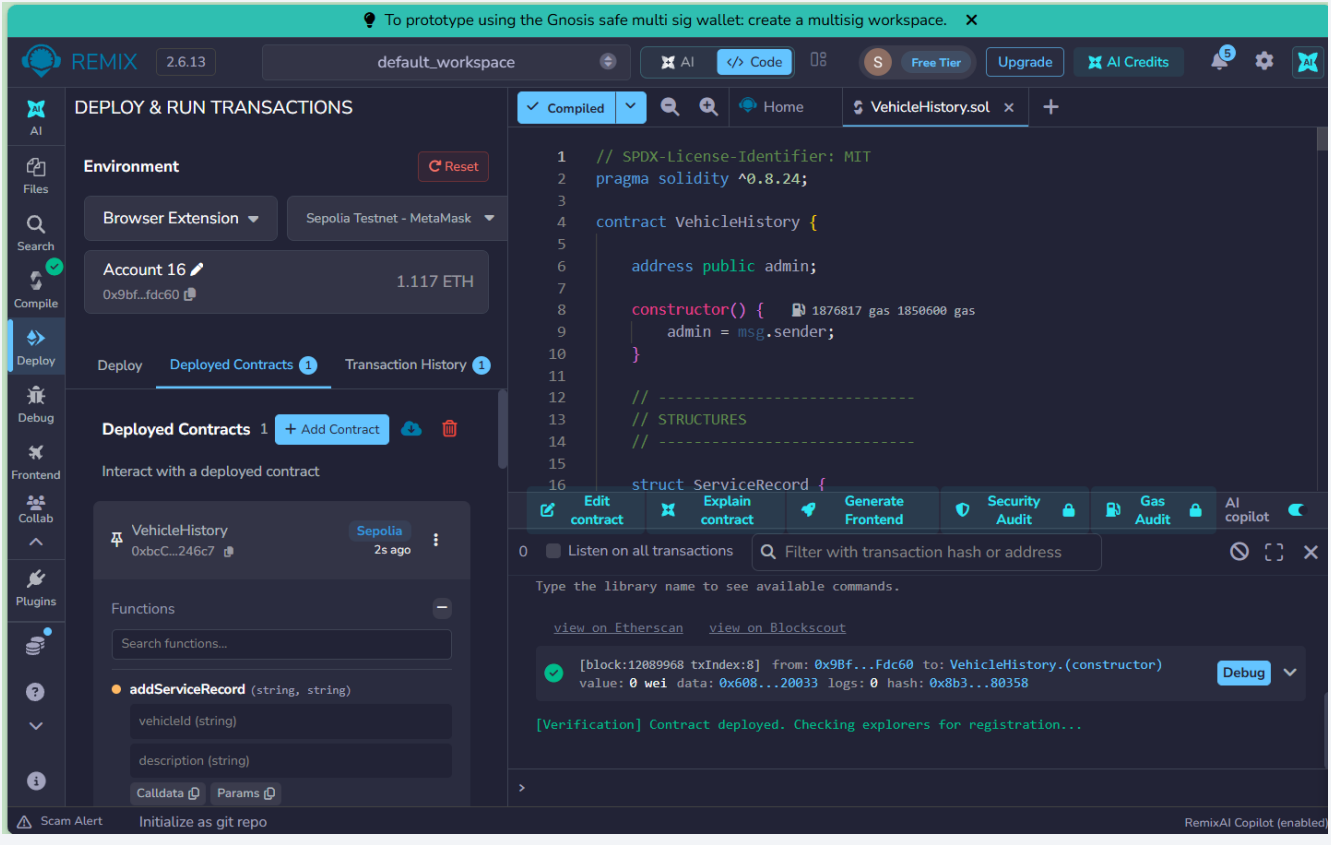
Test Case	Action	Expected Result	Observed Result
1	Register vehicle MH12AB1234	Vehicle is stored	Successful
2	Authorise service-centre wallet	Address becomes authorised	Successful

3	Add service record from authorised wallet	Service record is stored	Successful
4	Read getVehicleDetails()	Vehicle information and owner returned	Successful
5	Read getOwnershipHistory()	Ownership records returned	Successful
6	Read getServiceHistory()	Service records returned	Successful
7	Attempt service record from unauthorised wallet	Transaction reverts	Correctly reverted
8	Attempt ownership transfer from non-owner	Transaction reverts	Correctly reverted
9	Transfer ownership from current owner	Ownership changes and history updates	Successful

During testing, an ownership-transfer attempt from a different wallet initially failed with the contract message 'Only current owner can transfer ownership'. The transfer succeeded after using the current owner's account, confirming that the access-control rule was functioning as intended.

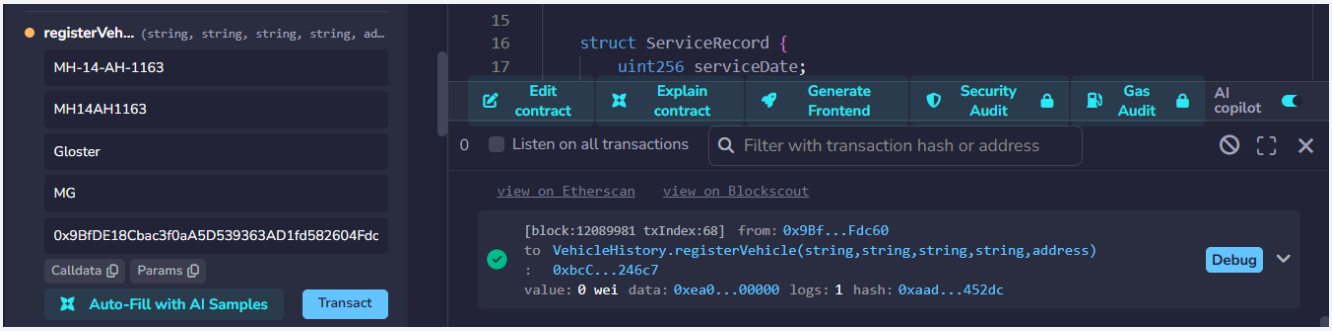
9. Screenshots

The following screenshots should be included in the final submission as evidence of the implementation and testing. The placeholders below are intentionally provided so the final report can use the actual project screenshots captured during the demonstration.



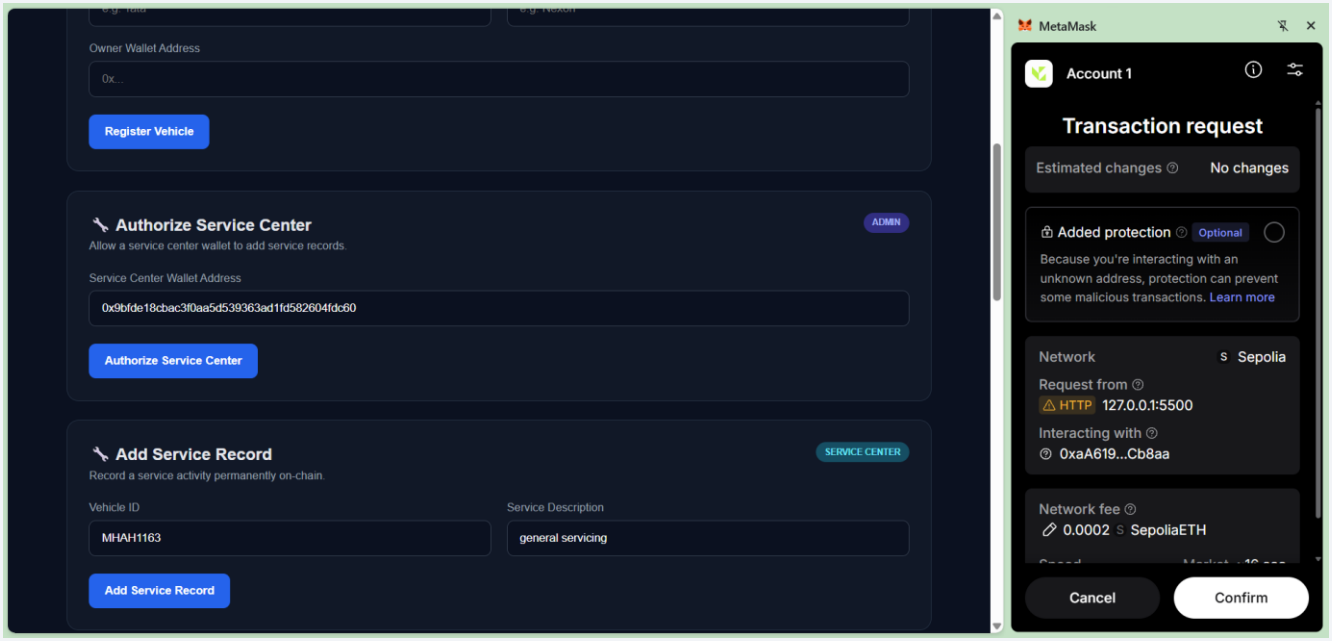
Remix IDE – Contract Compilation and Deployment.

Figure: Remix IDE – Contract Compilation and Deployment



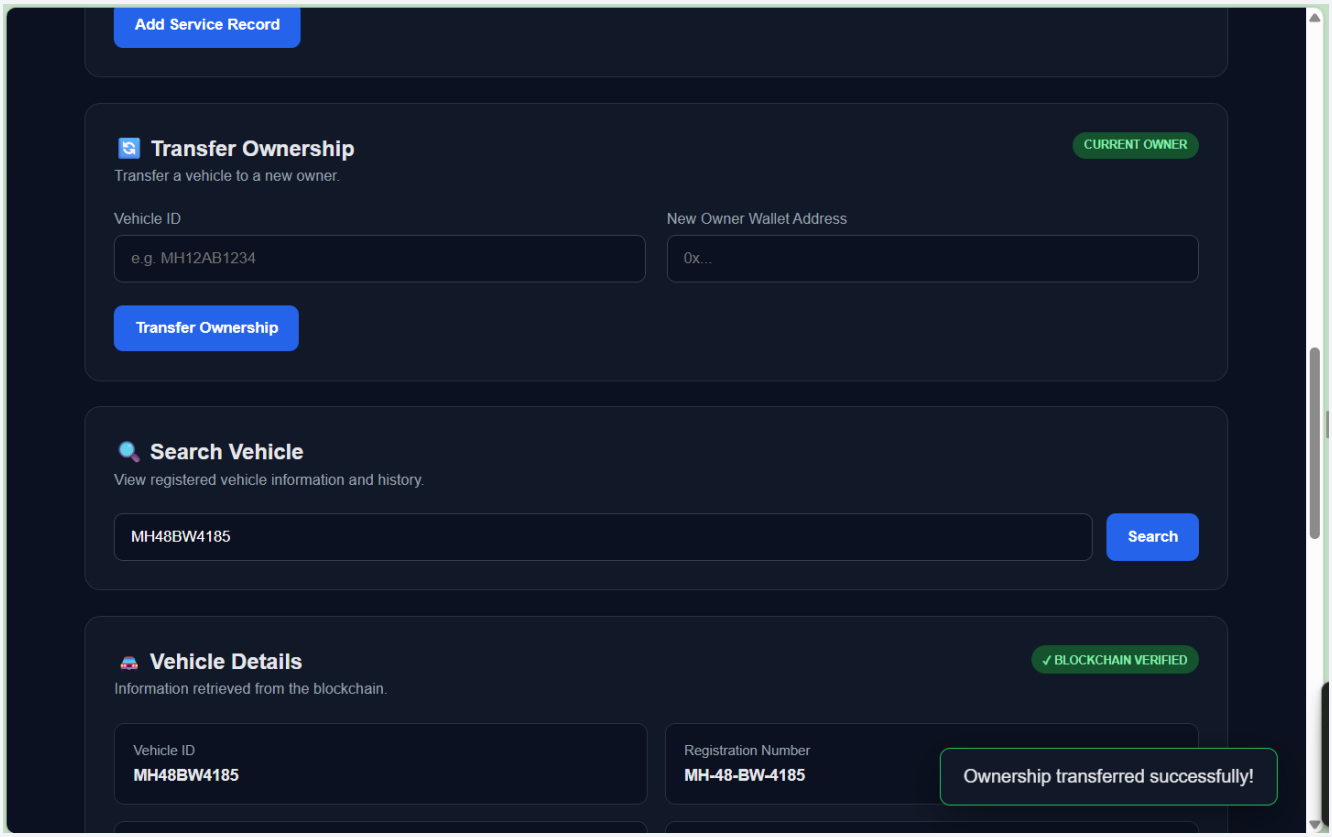
Vehicle Registration and Contract Interaction

Figure: Vehicle Registration and Contract Interaction



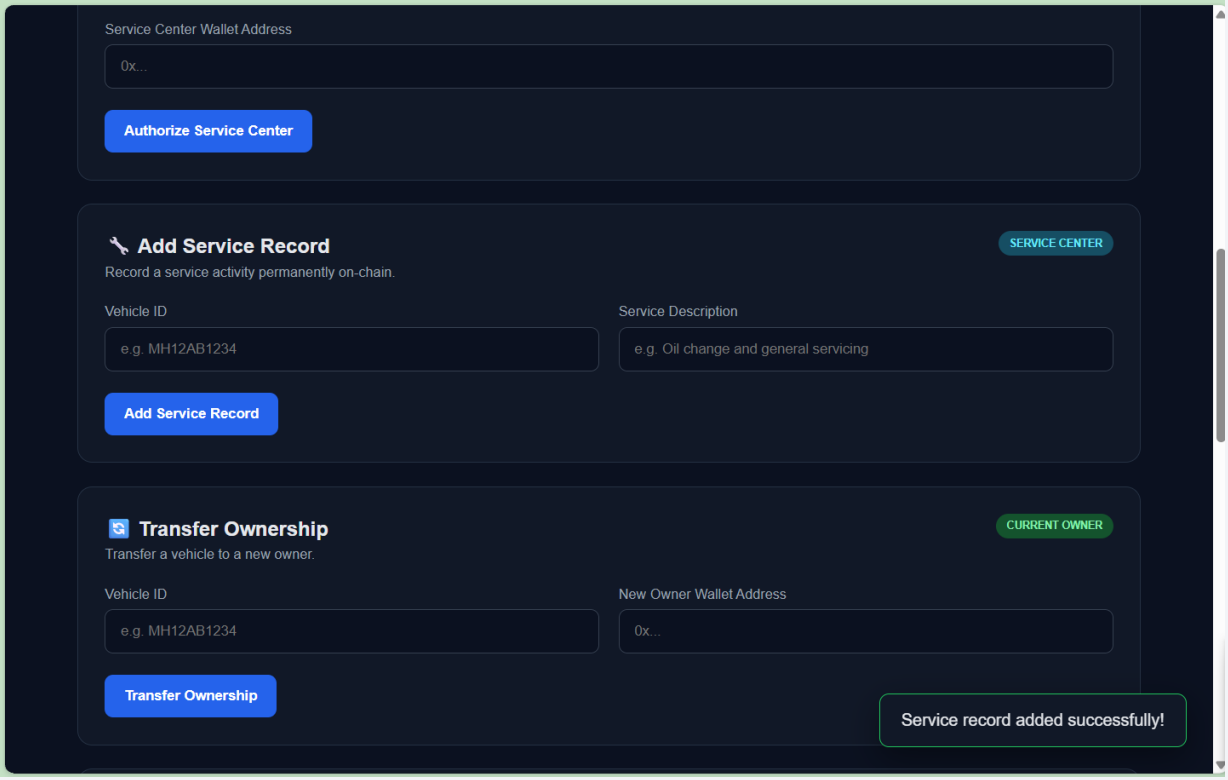
Service Centre Authorization and Service Record

Figure: Service Centre Authorization and Service Record



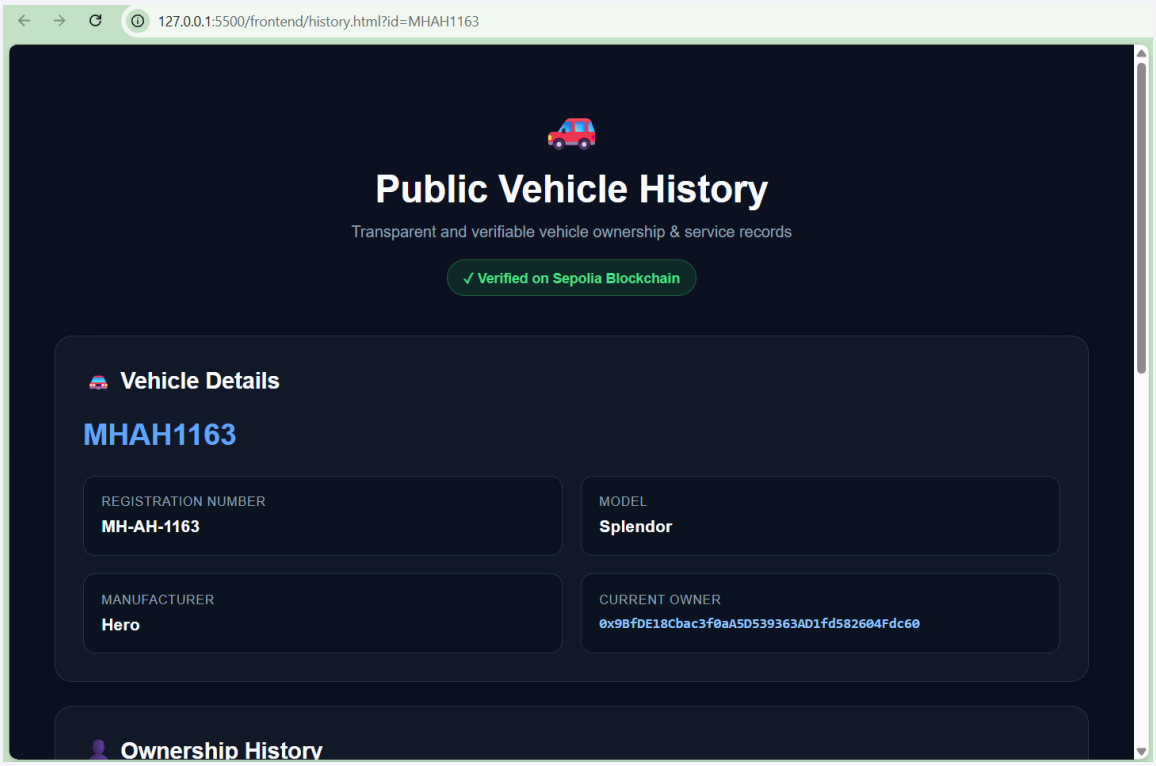
Ownership Transfer

Figure: Vehicle Details / Ownership History



Service History

Figure: Service History



Frontend Vehicle History / Public History Page

Figure: Frontend Vehicle History / Public History Page

10. Challenges Faced

Access-control debugging: The addServiceRecord() function initially reverted because the calling address was not authorised. The issue was resolved by authorising the correct service-centre wallet.

Wallet/account management: Different functions require different roles. The correct MetaMask/Remix account must be selected before sending a transaction.

Ownership transfer validation: A transfer attempt from a wallet other than the current owner reverted as expected. Testing required switching to the actual current-owner account.

Frontend-to-contract integration: The frontend requires the deployed contract address and the correct ABI. Updating these values is necessary whenever the contract is redeployed.

Public history page: The public history page requires a vehicle ID to load the corresponding blockchain data. A blank page can occur if the page does not contain the required frontend HTML/JavaScript implementation.

Testnet deployment: Moving from Remix VM to a browser-wallet environment introduces wallet, network and transaction-confirmation steps that must be handled correctly.

11. Security and Limitations

- Only the administrator can authorise service-centre addresses.
- Only authorised service centres can add service records.
- Only the current owner can transfer ownership.
- Previously recorded blockchain data is not intended to be edited or deleted by ordinary users.
- The project uses wallet addresses as identities; a wallet address by itself does not prove a person's real-world identity.
- The current implementation is a project/testnet demonstration and is not a replacement for official vehicle-registration authorities.
- A production deployment would require stronger identity verification, privacy considerations, security auditing and integration with trusted vehicle databases.

12. Conclusion

The Blockchain-Based Vehicle Ownership and Service History project demonstrates how smart contracts can be used to maintain a transparent and verifiable history for used vehicles. The system covers the core PS-12 requirements: vehicle registration, unique vehicle identification, current ownership, authorised service-centre records, ownership transfer, vehicle search and historical data retrieval.

Testing in Remix confirmed that the contract can store and retrieve vehicle, ownership and service information and that access-control rules prevent unauthorised operations. The frontend extends the smart contract into a practical dApp interface, while a vehicle-specific public history page provides a convenient way to present the recorded information.

13. Future Enhancements

- Generate a QR code that opens the public history page for a vehicle.
- Add a richer service record model containing mileage, parts replaced, cost and service-centre name.
- Add a search interface with vehicle ID or registration number.
- Add role-based dashboards for administrators, owners and service centres.
- Deploy on a production blockchain after security auditing and cost analysis.
- Add cryptographic document hashes for invoices, inspection certificates and other supporting documents.
- Introduce stronger identity and verification mechanisms for real-world service centres and owners.

14. Technical Details

Item	Project Detail
Smart Contract	VehicleHistory.sol
Programming Language	Solidity
Development Environment	Remix IDE
Wallet Integration	MetaMask / Browser Extension
Frontend	HTML, CSS and JavaScript
Contract Integration	Contract ABI + deployed contract address
Core Data	Vehicle details, ownership history and service history
Demo Vehicle ID	MH12AB1234
Example Vehicle	Nexon, Tata
Example Service	Oil change and general vehicle servicing